4.6 Packages and reproducible installs

Chapter 20 finishes by installing and using a third-party library. Since some students have limited access to the terminal, we will install all of the packages that we anticipate needing throughout the course. We will use them later in the course and they will just sit unused in the `vendor` directory until we need them.

Find and require the packages

Visit symfony/dotenv on Packagist, phpmailer/phpmailer on Packagist, and markrogoyski/math-php on Packagist. Identify the package name, description, requirements, and source/documentation links. Packagist is Composer's default package registry; Composer uses its metadata to locate releases and resolve dependencies.

At the repository root, run:

```
composer require symfony/dotenv phpmailer/phpmailer markrogoyski/math-php
```

Composer selects compatible releases, records constraints in `require`, writes exact resolved versions to `composer.lock`, installs code in `vendor/`, and regenerates the loader. Runtime dependencies belong in `require`, not `require-dev`. Inspect the resulting changes before proceeding. If platform requirements fail, check the CLI PHP version and extensions rather than bypassing them.

Package identifiers and PHP namespaces are different:

Package identifier	Class used in 4.7	Purpose
`symfony/dotenv`	`Symfony\Component\Dotenv\Dotenv`	Read configuration from a .env file
`phpmailer/phpmailer`	`PHPMailer\PHPMailer\PHPMailer`	Build email messages and support delivery

The packages provide their own autoload rules. Do not map their namespaces into our `src/` or copy their code there.

Let's add a few more packages that we'll be needing throughout the course. Although these aren't needed now and could be installed later, since some students will have limited access to the terminal and they won't have any impact until we use them, we'll install them now. As we did above, you can require multiple packages at once by separating them with a space. Run the `composer require` command with the following packages:

```
twig/twig
monolog/monolog nvanheuverzwijn/monolog-logdna mattsmithdev/pdo-crud-for-free-
repositories doctrine/orm symfony/cache guzzlehttp/guzzle
```

You can search for these packages on Packagist to learn more about them.

Inspect the completed configuration

This is a complete illustrative `composer.json` for PHP 8.4. Keep the compatible constraints generated by your require command if they differ; do not replace existing dependencies or metadata merely to match this example.

```
{
    "name": "student/class4-app",
    "description": "CIS 334 class 4 application",
    "type": "project",
    "license": "proprietary",
    "require": {
        "php": "^8.4",
        "symfony/dotenv": "^7.3",
        "phpmailer/phpmailer": "^6.10",
        "markrogoyski/math-php": "^2.0",
        "twig/twig": "^3.0",
        "monolog/monolog": "^3.0",
        "nvanheuverzwijn/monolog-logdna": "^4.0",
        "mattsmithdev/pdo-crud-for-free-repositories": "^2.0",
        "doctrine/orm": "^3.0",
        "symfony/cache": "^8.0",
        "guzzlehttp/guzzle": "^8.0"
    },
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    },
    "scripts": {
        "hello": "echo Hello class 4",
        "demo": "@php public/class4/index.php"
    }
}
```

These constraints illustrate version ranges, not a claim about the latest release. `^7.3` allows versions from 7.3.0 up to, but not including, 8.0.0. `^6.10` allows 6.10.0 up to, but not including, 7.0.0. For a major version above zero, a caret keeps updates within that major version. Zero-major constraints have tighter rules. The lock file records the exact selected versions, including indirect dependencies.

Distinguish daily commands

Command	Use
`composer require vendor/package`	Add a dependency and resolve/install it
`composer install`	Reproduce versions from an existing lock file; resolve and create one if absent

Command	Use
`composer update`	Deliberately resolve allowed versions again and change the lock file
`composer dump-autoload`	Regenerate loading metadata without upgrading packages
`composer show symfony/dotenv`	Inspect an installed package and version

Run `composer validate`, `composer show symfony/dotenv`, and `composer show phpmailer/phpmailer`. Then run `composer demo` to verify our own classes still work.

Add `/vendor/` and `/.env` to the root `.gitignore`, preserving existing entries. Commit `composer.json` and `composer.lock` for this application. When your instructor clones the repository and runs `composer install`, it will reconstruct your environment's dependencies using the versions specified in the lock file.

Before we are done, open the `composer.lock` file and inspect the `packages` array. Each package has a `name`, `version`, and `source` with a commit hash. The lock file is authoritative; it is not a suggestion or a list of available versions. The lock file is not meant to be edited manually.

References: Composer basic usage and version constraints.